# Design of the Modbus communication through serial port in QNX operation system

Sen Xu, Haipeng Pan, Jia Ren and Jie Su
*Department of Automatic Control, Zhejiang Sci-Tech University*
*E-mail: pan@zstu.edu.cn*

## Abstract

*This paper presents a general design method on serial interface device driver, which is based on the analysis of Modbus protocol and QNX Operation System (QNX OS) device driver architecture. Further, the paper specifies serial port communication in C program under QNX OS, and tests its communication with SIEMENS S7-200 PLC. The test result has proved that the method is real-time, stable, and reliable.*

## 1. Introduction

QNX Operation System (QNX OS) is one of multitask real-time operation systems, which is built on microkernel and protected address space mechanism. The advantages of QNX OS are real-time, stable, and reliable. QNX OS has been applied in fields such as industry automation, aviation, automobile, telecom, and its excellent performances are approved in those fields. Modbus protocol is one of protocols in industry automation, which is used widely. Modbus is also adopted as standard communication protocol in lots of instruments such as Schneider Modicon M430 series PLC, Delta VFD-M series AC motor inverter, Yokogawa DX1000/DX2000 paperless recorders, Weinview MT8000 Human-Machine Interface (HMI), etc. Modbus is integrated by some manufacturers in their products, thus the products become open so that other devices can communicate with it. Since Modbus protocol isn't integrated in QNX OS, this paper presents a general design method on the serial port device driver, which is based on the analysis of QNX host interface and loom electronic let-off and take-up control system, furthermore, C program is specified on Modbus protocol through serial port communication in QNX OS.

## 2. QNX operation system

QNX OS is a distributed, multi-user, multitask real-time operation system (OS), which is a product of QNX Software Systems International Corporation in Canada. QNX is a UNIX-like operation system, which complies with the POSIX 1003.1-2001 (POSIX.1) standards, so it allows quick migration from Linux, UNIX, and other open source programs. QNX OS is made up of a microkernel (about dozens of KB) and some modules that can be custom-built. It is easy to tailor QNX OS to satisfy user's various requirements in practical application due to its high flexibility.

The real time performance of QNX OS mainly manifests in the interrupt response delay and the context switch delay under the QNX OS, further, those responses are microsecond-level in the common hardware platform (such as R527X MIPS, SA-1110 StrongARM, Samung 2410 etc.), thus rigorous real time requirements can be satisfied. Canadian Space Agency selected twenty from forty-eight RTOS (Real-Time Operating Systems) in the market, then they strictly evaluated those RTOS [1], the evaluation result which is based on the comprehensive performance is depicted in Table 1.

**Table 1 Comprehensive Ranking of RTOS**

| Operation system | Comprehensive ranking |
|---|---|
| QNX/NeutrinoTM | 1st |
| OS-9TM | |
| Precise/MQX™ | 2nd |
| OSE™ | |
| Delta OS™ | 3rd |
| RTEMS™ | |
| Lynx OS™ | |
| Integrity™ | |
| VxWorks™ | |
| Nucleus Plus™ | 4th |
| VRTX™ | |
| TTPos™ | |
| C Executive™ | |
| Emb OS™ | |
| CMX™ | |
| ECOS™ | 5th |
| uC/OS-II™ | |
| SuperTask™ | |
| AMX™ | |
| Cortex™ | 6th |

We can see from Table 1 that the comprehensive performance of QNX OS is excellent. QNX OS becomes

● Supported by the Science and Technology Projects of Zhejiang province (2006C31016 and 021101039)

978-0-7695-3290-5/08 $25.00 © 2008 IEEE
DOI 10.1109/CCCM.2008.271

an open source code OS now, thus the application will be more and more in the future.

## 3. The architecture of device driver in QNX OS

Just as in UNIX OS, all devices are handled by device files in QNX environment. The interfaces of QNX OS comply with POSIX.1 standards too. Generally speaking, the device drivers of QNX include two layers: HSL (hardware shielding/ abstraction layer) and HAL (hardware access layer). HAL provides the logic interface for the hardware, and HAL executes the hardware operation. Fig. 1 gives the architecture of device drivers in QNX OS [2].

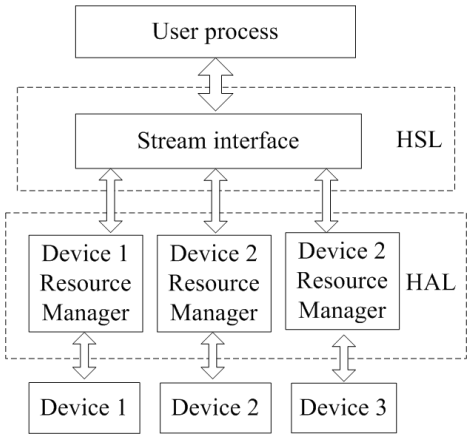


**Figure** 1. **Architecture of Device Drivers in QNX OS**

As Fig. 1 shows, HSL access device by calling the POSIX functions, thus program's portability and generality can be ensured. In addition, the HSL can be called as the stream interface too. Uniform framework architecture is used for device driver, which is defined as resource manager. A resource manager is a user-level server program that accepts messages from other programs and, optionally, communicates with hardware [3]. Resource managers are responsible for presenting an interface to various types of devices. Resource manager is a common process which can be optionally started, configured or halted, therefore hot-swappable hardware is allowable.

The messages from HSL are received by resource managers, and corresponding operations are performed in HAL layer, in the end, the result is returned to the HSL. The communication between device drivers and device is implemented by some functions such as open ( ), read ( ), write ( ), evctl ( ), and ioctl ( ), which are supplied by QNX OS for hardware.

Resource managers access the system kernel through message mechanism. As kernel interface and user layer is separate, the reliability of this architecture is high. Adding resource managers in QNX won't affect any other part of the OS—the drivers are developed and debugged like any other application. And since the resource managers are run in their respective protected address space, a bug in a device driver won't cause the entire OS to shut down. This architecture brings convenience in design and debug.

## 4. Modbus protocol introduction

Modbus protocol developed by Modicon Corporation, is widely applied in the industrial control network, which has become one of prevalent open industrial standards. Modbus is a master-slave protocol, which can be used to connect other instruments made in different manufactory into a network, and centralized monitoring can be achieved. Modbus describes the process a controller communicates with another device, how master device query slave devices, how slave devices respond to requests from the master device, and how errors will be detected and reported. It establishes a common format for the layout and contents of message fields.

The characteristics of Modbus are as follows: RS232/422/485 standards can be adopted by the hardware interface, and the slave nodes can be read or written by the master node through some simple message frame in the Modbus network. When the master queries, the slaves execute the requested action, and respond to the master by supplying the requested data. There are no responses if master broadcast queries. If no error occurs in the communication, the response function code is consistent with the query function code and the information about slave is included in data field. Otherwise, the function code is replaced to indicate error, and correlative error information is included in the data field.

Modbus has two transmission modes: ASCII and RTU, a Modbus message is placed by the transmitting device into a frame that has a known beginning and ending point [4]. Typical ASCII message frame and RTU message frame are specified in the literature [4]. The main advantage of ASCII mode is that it allows time intervals of up to one second to occur between characters without causing an error. The main advantage of RTU mode is that its greater character density allows better data throughput than ASCII mode for the same baud rate. This paper adopts the RTU mode for high efficiency.

The message frame of device in the Modbus network can be set as either RTU Mode or ASCII mode, and furthermore, all devices in the network should be the same transmission mode, the same message frame and the same serial parameter.

## 5. Design of the serial port communication and Modbus application in QNX OS

Electronic let-off and take-up is applied in loom, which can not only improve the quality of fabric, but also eliminate fabric defects. The weft density can be regulated discretionarily by using the techniques, thus fabric variety update may become faster. The electronic let-off and take-up systems are highly sensitive and its structures are simple, therefore this is one of the important development trends [5]. Rapier loom is studied by this paper, whose electronic let-off and take-up system structure diagram can be seen from Figure 2.

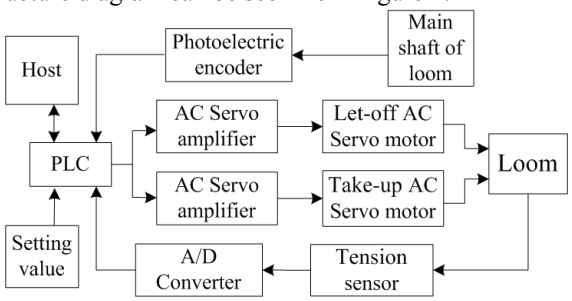


**Figure 2. Rapier Loom Electronic Let-Off And Take-Up Systems Structure Diagram**

After synthesizing the warp tension setting value, warp tension actual value and spindle speed, PLC regulates the let-off AC servo motor speed through controlling the AC servo amplifier to keep the warp tension uniform and constant. At the same time, PLC calculates the take-up AC servo motor speed based on setting weft density and spindle speed. Then PLC regulates the take-up AC servo motor speed so as to prove the completed fabric is taken up. Here PLC is Siemens S7-200, and the host is QNX OS based on Samsung 2240 chip. Since host communicates with S7-200 PLC through Modbus protocol, a host can monitor dozens of looms at the same time.

Peripheral generally consists of character device and block device in QNX OS. Since it's common to run multiple character devices, a module called io-char has been designed as a family of drivers and a library to maximize code reuse in QNX OS. The io-char module contains all the codes to support POSIX semantics on the device. It also contains a significant amount of codes to implement character I/O features beyond POSIX but desirable in a real-time system [6]. Since this code is in the common library, all drivers inherit these capabilities. In operation, io-char module can be invoked by driver, therefore designing the driver become more convenient. The drivers themselves are just like any other QNX OS process and can run at different priorities according to the nature of the hardware being controlled and the client's requesting service. Once a single character device is running, the memory cost of adding additional devices is minimal, since only the code to implement the new driver structure would be new [6].

The io-char library manages the flow of data between an application and the device driver. Data flows between io-char and the driver through a set of memory queues associated with each character device. Each device has three queues: raw input queue, canonical queue and output queue. Each queue is implemented using a first-in, first-out (FIFO) mechanism [6]. Figure 3 depicts the architecture of serial port.

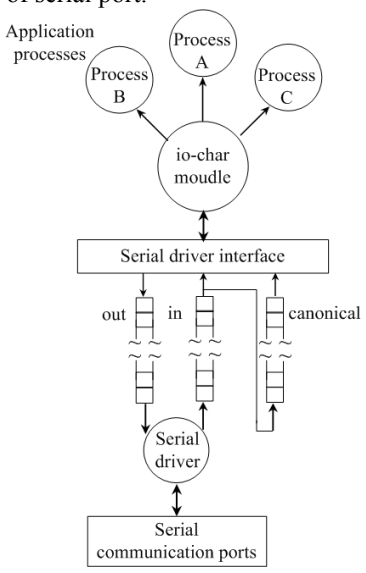


**Figure 3. Architecture of serial device I/O interface**

Received data is placed into the raw input queue by the driver and is consumed by io-char module only when application processes request data. The io-char module places output data into the output queue to be consumed by the driver as characters are physically transmitted to the device. Each serial port on QNX systems has one or more device files (files in the /dev directory) associated with it, see Table 2.

**Table 2. Serial Port Derives File**

| | Port 1 | Port 2 |
|---|---|---|
| Device file | /dev/ser1 | /dev/ser2 |

The following steps describe a serial port communication method in QNX OS. First, open the serial port device file and save the old serial port settings, second, enable receive and configure the serial port parameter [3][7][8], third, deal with the communication protocol (e.g. packet the data), fourth, write the data or data packet to the serial port ,fifth, read the data from serial port, last, terminate the process if no errors occur. Fig. 4 depicts the flow chart.

This paper uses serial port to communicate with

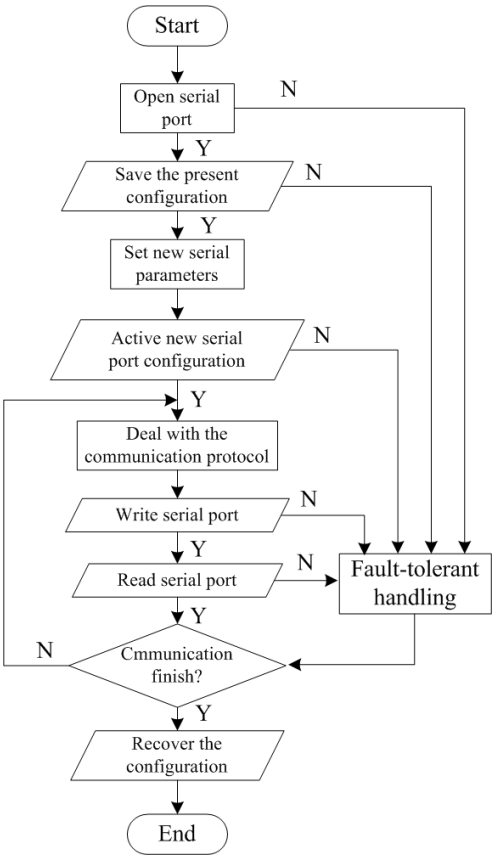


**Figure 4**. **Serial Port Communication Flow Chart in QNX System**

S7-200 through Modbus protocol, which is Modbus RTU mode. The serial port adopts raw input mode, nonblock, baud rate is 9600, 8 data bits, 1 stop bit, no parity bit. The function of the following program is to read the output coil state from S7-200 CPU 226. The main program is like this:

```
   #include <stdio.h>      /* Standard input/output definitions */
   #include <unistd.h>     /*UNIX standard function definitions */
   #include <fcntl.h>      /* File control definitions */
   #include <termios.h>/*POSIX terminal control definitions */
   #include <errno.h>      /* Error number definitions */
   #include <strings.h>    /* bzreo() */
   #include <stdlib.h>     /* exit() _exit() */
   Int MODBUS_readcoil(unsigned int dev_addr,unsigned int fc_num,unsigned int startno,unsigned int endno,unsigned char * buf )
   {
        buf[0]=dev_addr;
        buf[1]=fc_num;
        buf[3]=(unsigned char)startno;
        buf[2]=(unsigned char)(startno>>8);
        int coilnum=endno-startno+1;
        buf[5]=(unsigned char)coilnum;
        buf[4]=(unsigned char)(coilnum>>8);

        short sCRC = CalCRC(buf, 6);      /* calculate CRC*/
        buf[7] = (char)sCRC;
        buf[6] = (char)(sCRC>>8);
        return (1);
   } /* packet data in term of Modbus protocol */

   main( )
   {
        int fd,temp_crc;
        unsigned char readbuff[256];
        unsigned char writebuff[8];
        struct termios old_termi,new_termi;
        for(int i=1;i<=5;i++)
        {
          fd=open("/dev/ser1",  O_RDWR|O_NOCTTY);
        /* open serial port device file */
          if (fd>=0)
          break;  /* successfully open serial port */
          if ( fd<0 &&i==5)
          {
           printf("open the serial port failed! errno is: %d\n", errno);
        _exit(EXIT_FAILURE);  /* open serial port failed*/
          }
         delay(500);       /* try it again after half a second */
        }
        if ( tcgetattr( fd, &old_termi) != 0)  /* save the old settings */
        {
   printf("get the terminal parameter error! errno is: %d\n", errno);
        exit(EXIT_FAILURE);
        }
        bzero(&new_termi,sizeof(new_termi));  /* clear the structure */
        new_termi.c_cflag |= (CLOCAL|CREAD);  /* receive enable，ingor the state of modem */
        new_termi.c_lflag&=~(ICANON|ECHO|ECHOE);       /* set the raw input mode */
        new_termi.c_oflag&=~OPOST;
        /* set the raw output mode */
        new_termi.c_cc[VTIME] = 5;       /* set the overtime is 0.5s */
        new_termi.c_cc[VMIN] = 7;        /* the least num of bytes is 7 */
        cfsetispeed(&new_termi, B9600);  /* set input baud rate */
        cfsetospeed(&new_termi, B9600);  /* set output baud rate */
```

```
        new_termi.c_cflag &= (~CSIZE);  /*  8  data
bits*/
        new_termi.c_cflag |= CS8;

        new_termi.c_cflag &= ~PARENB;  /* no parity
bit */
        new_termi.c_iflag &=~ ISTRIP;
        new_termi.c_cflag &= ~CSTOPB;  /* 1 stop bit
*/
        tcflush(fd, TCIOFLUSH);          /* flush the
input and output stream  */
        if (tcsetattr(fd, TCSANOW, &new_termi)!= 0)
        /* active the serial configurations */
        {
                printf("Set  serial  port  parameter
error!\n");
                exit(EXIT_FAILURE);
        }
        MODBUS_readcoil(2,1,0,15,writebuff);
        if (write(fd, writebuff, 8)==-1)      /* send the
packeted data to serial port */
        {
                printf("write serial port error!\n");
                exit(EXIT_FAILURE);
        }
        if (read(fd, readbuff, 7)==-1)      /*  receive
data from serial port，and save it */
        {
                printf("read serial port error!");
                exit(EXIT_FAILURE);
        }
        for(int i=0;i<7;i++)
        {
                printf("%02X ",readbuff[i]);
        /* display the received data */
        }
        printf("\n");
        tcsetattr(fd, TCSANOW, &old_termi);
        /* recover the old settings */
        close(fd);          /* close serial port */
    }
```

The hardware platform used in the test is S7-200 CPU 226, and QNX edition is QNX 6.2.1. All of the functions in the program comply with POSIX.1 standards for portability. The subprogram *CalCRC( )* is omitted here. The C program is extended in this paper by adding fault tolerant mechanism and all the Modbus functions that S7-200 PLC supports. Then, a long-term rigorous test is conducted on the extended program in practice for a month, and the test result is Table 3.

The test result shows that its bit error ratio is very low, performance is stable, reliability is high and the systems response is very fast (microsecond level), therefore this design can meet actual demand.

**Table 3. Test Result Of System Communication**

| Function code | QNX Sending data (hexadecimal) | PLC Response data (hexadecimal) | Bit error |
|---|---|---|---|
| 1 | 02,01,00,00,00, 08,3D,FF | 02,01,01,F0, 51,88 | 10E-8 |
| 2 | 02,02,02,00,00, 10,79,B5 | 02,02,02,F0, 00,B9,B8 | 10E-8 |
| 3 | 02,03,00,04,00, 02,85,F9 | 02,03,04,00,00, 00,00,C9,33 | 10E-9 |
| 4 | 02,04,00,00,00, 02,71,F8 | 02,04,04,00,00, 00,00,C8,84 | 10E-8 |
| 5 | 02,05,00,00,FF, 00,8C,09 | 02,05,00,00,FF, 00,8C,09 | 10E-8 |
| 6 | 02,06,00,02,00, 0F,68,3D | 02,06,00,02,00, 0F,68,3D | 10E-9 |
| 15 | 02,0F,00,00,00, 10,02,FF,FF,F7,60 | 02,0F,00,00,00, 10,54,34 | 10E-8 |
| 16 | 02,10,00,00,00,02,04, AB,CD,23,45,95,F3 | 02,10,00,00,00, 02,41,FB | 10E-7 |

## 6. Conclusion

The proposed method has successfully accomplished the serial port communication based on Modbus protocol in QNX OS. Its communication is real-time, reliable and stable. In addition, Multiplexing that multi-process share one serial port can be achieved by calling *select ( )* function. The method in this paper can be extended to CAN, Profibus, and other protocols in QNX OS environment, so that QNX OS can communicate with more instruments.

## 7. References


[1] P. Melanson, S. Tafazoli, ”A selection methodology for the RTOS market,” *Data Systems in Aerospace conference*, Prague, June 2003..

[2] J. Q. Xu, J. F. Huang and Y. P. Li, “Programing device drivers for QNX OS,” *Computer Engineering*, vol. 29, no. 12, pp. 176-178, July 2003.

[3] *QNX Neutrino Realtime Operating System Programmer’s Guide*, QNX Software Systems GmbH & Co. KG., electronic edition published, 2007.

[4] *Modicon Modbus Protocol Reference Guide*, Rev. J, Modicon Inc., 1996.

[5] Z. N. Cheng, Z. L. Jiang, J. C. Zhan, and T. F. Chen, ”Electronic let-off and take-up control system based on PLC,” *Journal of Textile Research*, vol. 27,no. 10, pp. 102-104, October 2006.

[6] *QNX Neutrino Realtime Operating System System Architecture*, QNX Software Systems GmbH & Co. KG.， electronic edition published, 2007.

[7] M. R. Sweet, ”Serial programming guide for POSIX operating systems,” 5th ed., 2005.

[8] G. Frerking, “Serial programming howto,” Rev. 1.01, 2001.